TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

**KHOA TOÁN - TIN HỌC**

---




# BÁO CÁO THỰC HÀNH

Môn học: Xử lý dữ liệu lớn

**Bài thực hành L05**

# PySpark cơ bản: DataFrame, Spark SQL, kế hoạch thực thi và Spark UI

Sinh viên thực hiện: Nguyễn Thị Huyền Trang
Mã số sinh viên: 23280092
Lớp: 23KDL

TP. Hồ Chí Minh, Tháng 10 năm 2026

# Mục lục

**1 Mục tiêu và môi trường** **2**

**2 Thực hiện và kết quả** **4**

2.1 Đọc dữ liệu và lazy evaluation . . . . . 4
2.2 Làm sạch dữ liệu và chế độ ANSI . . . . . 5
2.3 Join với bảng khu vực và tổng hợp . . . . . 6
2.4 Hàm cửa sổ . . . . . 8
2.5 Spark SQL và kế hoạch vật lý . . . . . 10
2.6 Spark UI . . . . . 12
2.7 Ghi Parquet phân vùng và partition pruning . . . . . 20

**3 Trả lời câu hỏi checkpoint** **22**

3.1 Vì sao tệp 48 MB lại có 4 partition đầu vào? . . . . . 22
3.2 Dòng nào là transformation, dòng nào là action, và sao dựng plan nhanh vậy? . 23
3.3 `by_borough.show()` sinh bao nhiêu job, stage, và stage bị cắt ở đâu? . . . . . 23
3.4 Vì sao năm cột có cùng 140,162 NULL, và nên xử lý thế nào? . . . . . 23
3.5 `HashAggregate` partial trước `Exchange` làm gì, ứng với khái niệm nào ở L02? . 24
3.6 Vì sao bảng khu vực được broadcast, và khi nào Spark chuyển sang sort-merge join? . . . . . 24
3.7 `cache()` thay đổi gì trong plan và UI, và khi nào không nên cache? . . . . . 24
3.8 Vì sao cần `repartition("pickup_date")` trước `partitionBy("pickup_date")`? . . . . . 25

**4 Kết luận** **25**

**Tài liệu tham khảo** **25**

**Phụ lục: Trích đoạn mã nguồn** **26**

# Danh mục hình

1 Tab Environment của Spark UI: Java 17.0.20.1, Scala 2.13.17, `spark.driver.memory=2g`, event log được bật. . . . . 3
2 Đọc dữ liệu: schema 19 cột, 2,964,624 dòng, 4 partition; dựng ba transformation mất 0.096 s, `count()` mất 0.28 s. . . . . 5
3 Đếm NULL theo cột và số dòng vi phạm từng luật làm sạch; giữ lại 2,867,786 dòng (96.73 %). . . . . 6
4 Chế độ ANSI: `try_divide` trả NULL, phép chia thường ném lỗi `[DIVIDE_BY_ZERO]`. . . . . 6
5 Kết quả join hai lần với bảng khu vực, tổng hợp theo quận và theo giờ đón. . . 8

6 Hàm cửa sổ: top-3 khu vực đón mỗi quận và chuỗi theo ngày với `lag`, tổng lũy kế. 10
7 Kết quả Spark SQL trùng khớp DataFrame và cây kế hoạch vật lý của `by_borough`. 11
8 Phần kế hoạch thể hiện `HashAggregate` partial, `Exchange hashpartitioning` và `HashAggregate` final. . . . . 12
9 Tab Jobs: mỗi action sinh một hoặc nhiều job. . . . . 13
10 Tab SQL/DataFrame: 21 truy vấn đã hoàn thành cùng thời lượng và danh sách job. . . . . 14
11 Đồ thị kế hoạch truy vấn 10 trong tab SQL: 2,867,786 dòng vào, 24 dòng ra khỏi `HashAggregate` partial. . . . . 16
12 Job 21: một stage, 4/4 task, Input 347.3 MiB từ cache, Shuffle Write 2.3 KiB. . 17
13 Job 22: stage 28 bị bỏ qua, stage 29 chạy 1 task với Shuffle Read 2.3 KiB. . . . 18
14 Tab Stages: 60 stage hoàn thành, 28 stage bị bỏ qua, kèm Input và Shuffle Read/Write. . . . . 19
15 Tab Storage: `clean` được cache 4/4 partition, 100 %, 347.3 MiB trong bộ nhớ. . 20
16 Ghi Parquet phân vùng, đọc lại ngày 2024-01-15 (74,743 chuyến) và `PartitionFilters` trong kế hoạch. . . . . 21
17 Đồ thị truy vấn 18 (ghi Parquet): `Exchange` do `repartition` shuffle toàn bộ 2,867,786 dòng. . . . . 22

# Danh mục bảng

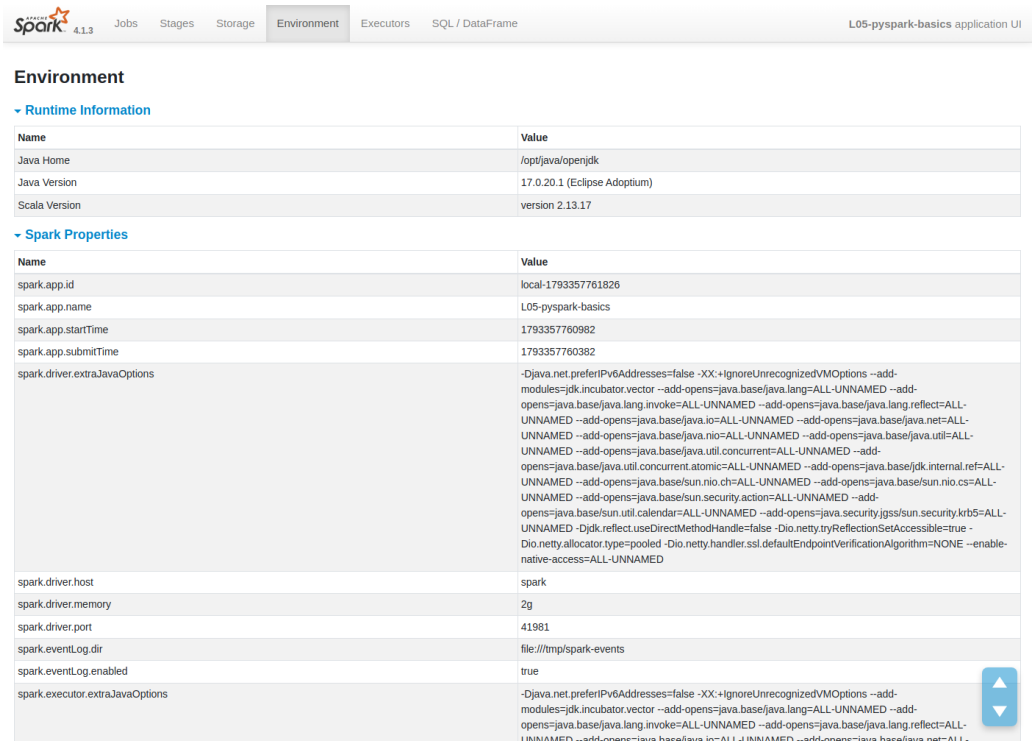
1 Môi trường thực nghiệm và cấu hình Spark. . . . . 3
2 Dữ liệu đầu vào. . . . . 4
3 Số dòng vi phạm từng luật làm sạch (một dòng có thể vi phạm nhiều luật). . . . 5
4 Tổng hợp theo quận đón sau làm sạch (`by_borough`). . . . . 7
5 Top-3 khu vực đón tại Manhattan và Queens. . . . . 9
6 Số chuyến và doanh thu theo ngày với `lag` và tổng lũy kế (10 ngày đầu). . . . . 9
7 Một số truy vấn trong tab SQL (log `ui_sql_queries.txt`). . . . . 14
8 Các job và stage của truy vấn 10 (`by_borough.show()`). . . . . 15
9 So sánh ghi Parquet phân vùng theo ngày có và không có `repartition`. . . . . 20

# 1 Mục tiêu và môi trường

Bài này dùng API DataFrame của PySpark để xử lý dữ liệu taxi vàng New York tháng 01/2024 [5]: đọc Parquet và CSV, kiểm chứng lazy evaluation, làm sạch dữ liệu (kèm chế độ ANSI của Spark 4), join với bảng khu vực, tổng hợp, dùng hàm cửa sổ, viết lại bằng Spark SQL, đọc kế hoạch vật lý cùng Spark UI và ghi Parquet phân vùng theo ngày. Cấu hình ở Bảng 1 lấy từ `conf/spark-defaults.conf` và được kiểm tra lại trong tab Environment (Hình 1); dữ liệu đầu vào ở Bảng 2.

**Bảng 1.** Môi trường thực nghiệm và cấu hình Spark.

| Thành phần | Giá trị |
|---|---|
| Phiên bản Spark | 4.1.3 (image Docker `spark:4.1.3-python3`) |
| Java / Scala / Python | Java 17 (17.0.20.1, Eclipse Adoptium) / Scala 2.13.17 / Python 3.10 |
| Chế độ chạy | `local[4]`: driver và 4 luồng task trong một JVM |
| Bộ nhớ driver | 2 GB (`spark.driver.memory=2g`) |
| `spark.sql.shuffle.partitions` | 8 (thay cho mặc định 200) |
| Khác | `spark.sql.session.timeZone=UTC`, `spark.sql.ansi.enabled=true` (mặc định Spark 4) |
| Event log, UI | event log ghi vào `file:///tmp/spark-events`; UI cổng 4040, giữ sống bằng `--hold 300` |




**Hình 1.** Tab Environment của Spark UI: Java 17.0.20.1, Scala 2.13.17, `spark.driver.memory=2g`, event log được bật.

**Bảng 2.** Dữ liệu đầu vào.

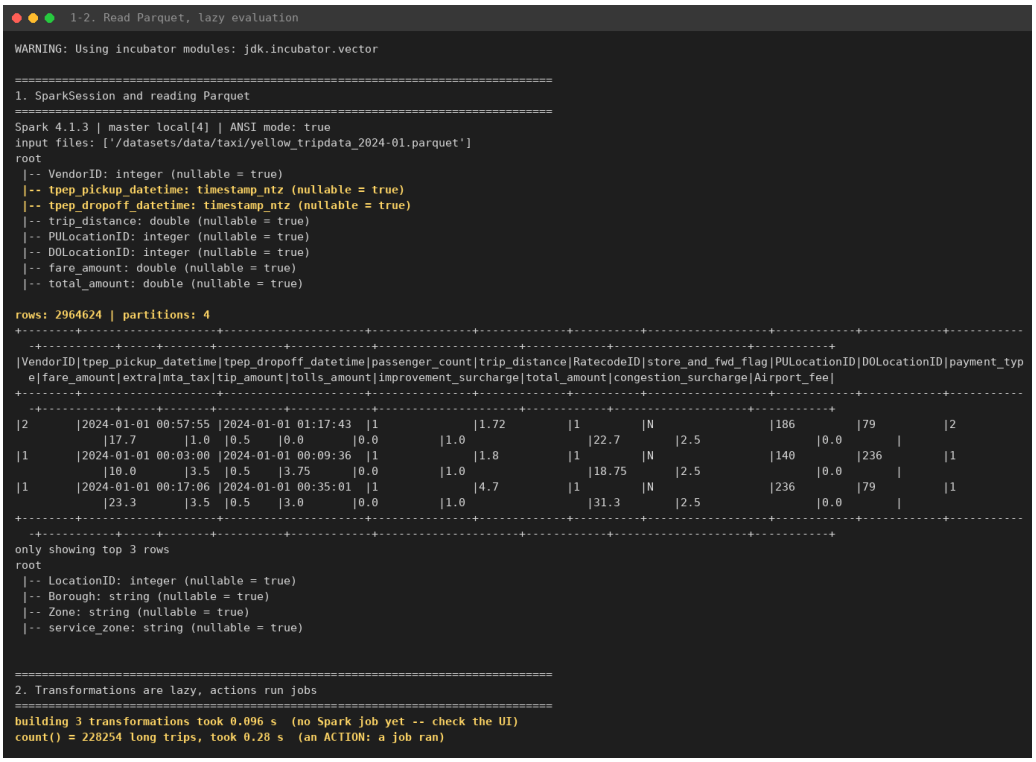
| Tệp | Định dạng | Kích thước | Số dòng | Số cột |
|---|---|---|---|---|
| `yellow_tripdata_2024-01.parquet` | Parquet | khoảng 48 MB | 2,964,624 | 19 |
| `taxi_zone_lookup.csv` | CSV có header | 12.0 KiB | 265 | 4 |

Toàn bộ các bước nằm trong `l05_basics.py`, chạy bằng `spark-submit` trong container. Để so sánh hai cách ghi Parquet mà không sửa mã, chúng tôi thêm cờ `--no-repartition`. Lần đầu chạy với cờ này (`01_run_no_repartition.txt`), lần thứ hai là lần chạy chính có `repartition` và `--hold 300` để chụp Spark UI (`03_run_with_repartition.txt`). Số liệu bên dưới lấy từ lần chạy chính nếu không ghi chú khác; hai lần cho kết quả giống hệt, trừ phần ghi Parquet và thời gian đo. Lần chạy chính kết thúc với mã 137 vì tiến trình bị dừng trong lúc `--hold`, khi mọi kết quả đã in xong, nên số liệu không bị ảnh hưởng.

# 2 Thực hiện và kết quả

## 2.1 Đọc dữ liệu và lazy evaluation

Spark đọc được 19 cột (hai cột thời gian kiểu `timestamp_ntz`), 2,964,624 dòng, chia thành 4 partition đầu vào (Hình 2). Ba transformation (`filter`, `withColumn`, `select`) được dựng trong 0.096 s và không sinh job nào; `count()` ngay sau đó trả về 228,254 chuyến dài hơn 10 dặm sau 0.28 s. Ở lần chạy không `repartition`, hai con số là 0.076 s và 0.26 s, cùng bậc độ lớn.




**Hình 2.** Đọc dữ liệu: schema 19 cột, 2,964,624 dòng, 4 partition; dựng ba transformation mất 0.096 s, `count()` mất 0.28 s.

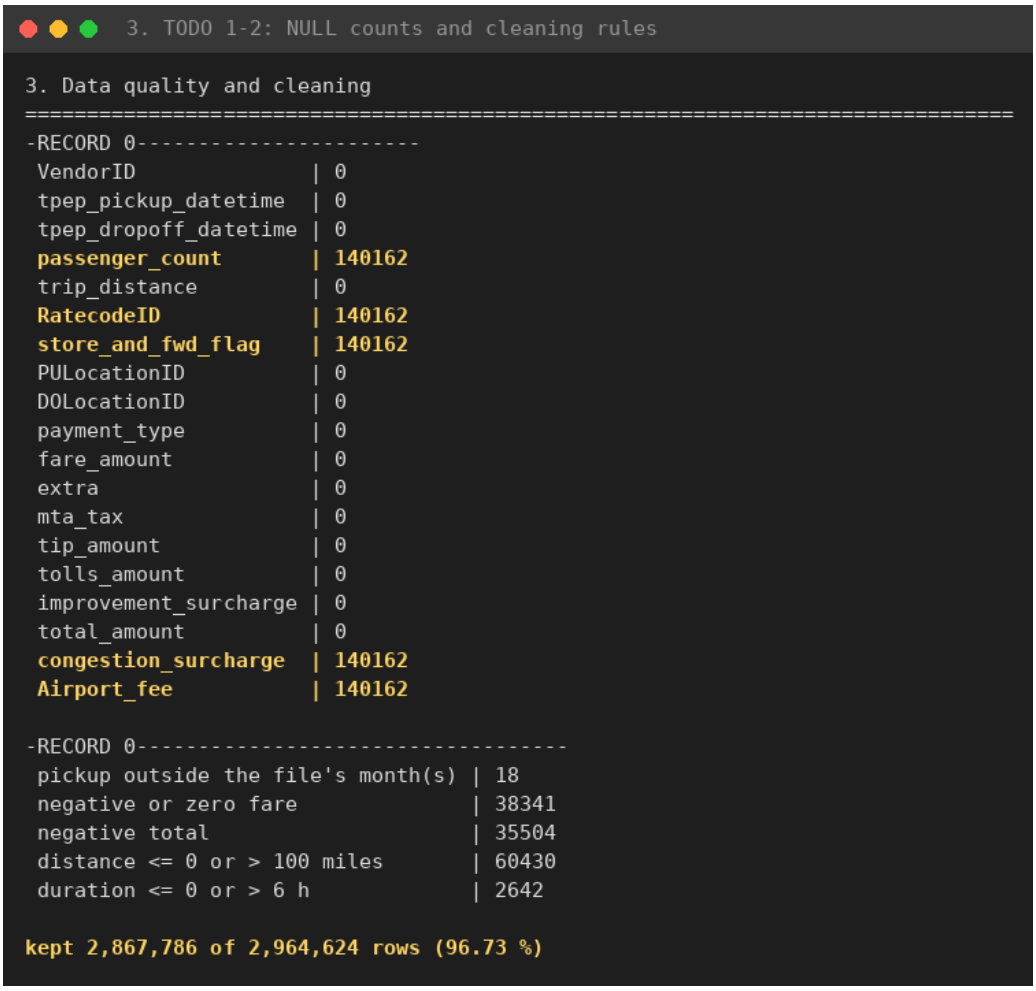
## 2.2 Làm sạch dữ liệu và chế độ ANSI

Số NULL của mọi cột được đếm trong một phép `select` duy nhất: 14 cột không có NULL, còn đúng năm cột `passenger_count`, `RatecodeID`, `store_and_fwd_flag`, `congestion_surcharge`, `Airport_fee` cùng có 140,162 NULL (Hình 3). Số dòng vi phạm từng luật được đếm bằng tổng hợp có điều kiện trong một lần quét (Bảng 3).

**Bảng 3.** Số dòng vi phạm từng luật làm sạch (một dòng có thể vi phạm nhiều luật).

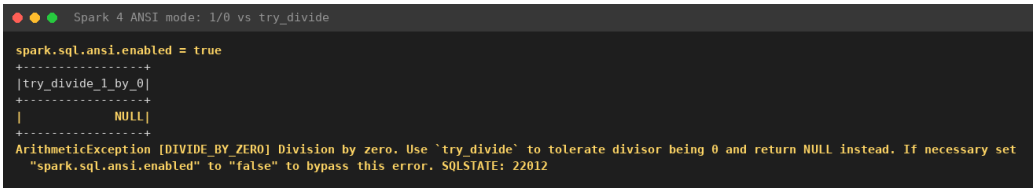
| Luật | Số dòng vi phạm |
|---|---|
| Thời điểm đón nằm ngoài tháng của tệp (2024-01) | 18 |
| Giá cước `fare_amount` ≤ 0 | 38,341 |
| Tổng tiền `total_amount` < 0 | 35,504 |
| Quãng đường ≤ 0 hoặc > 100 dặm | 60,430 |
| Thời lượng ≤ 0 hoặc > 6 giờ | 2,642 |
| **Giữ lại sau làm sạch** | **2,867,786 / 2,964,624 (96.73 %)** |

Tổng số dòng bị loại là 96,838, ít hơn tổng cột bên phải vì các luật chồng lên nhau: chuyến có giá cước âm thường cũng có tổng tiền âm. Dòng có NULL không bị loại mà được điền mặc định `passenger_count=1`, `RatecodeID=1`, `congestion_surcharge=0.0`, `Airport_fee=0.0`. DataFrame `clean` sau đó được `cache()`.




**Hình 3.** Đếm NULL theo cột và số dòng vi phạm từng luật làm sạch; giữ lại 2,867,786 dòng (96.73 %).

Từ Spark 4.0, ANSI mode bật mặc định nên chia cho 0 sẽ báo lỗi thay vì trả NULL như Spark 3 [3]; phép chia `tip_amount / fare_amount` thông thường sẽ làm hỏng cả job nếu gặp mẫu số bằng 0, nên tỷ lệ tip được tính bằng `F.try_divide`. Đoạn thử riêng (Hình 4) xác nhận `spark.sql.ansi.enabled = true`: `try_divide(1, 0)` trả NULL, còn `1/0` ném `ArithmeticException [DIVIDE_BY_ZERO]` với SQLSTATE 22012.




**Hình 4.** Chế độ ANSI: `try_divide` trả NULL, phép chia thường ném lỗi `[DIVIDE_BY_ZERO]`.

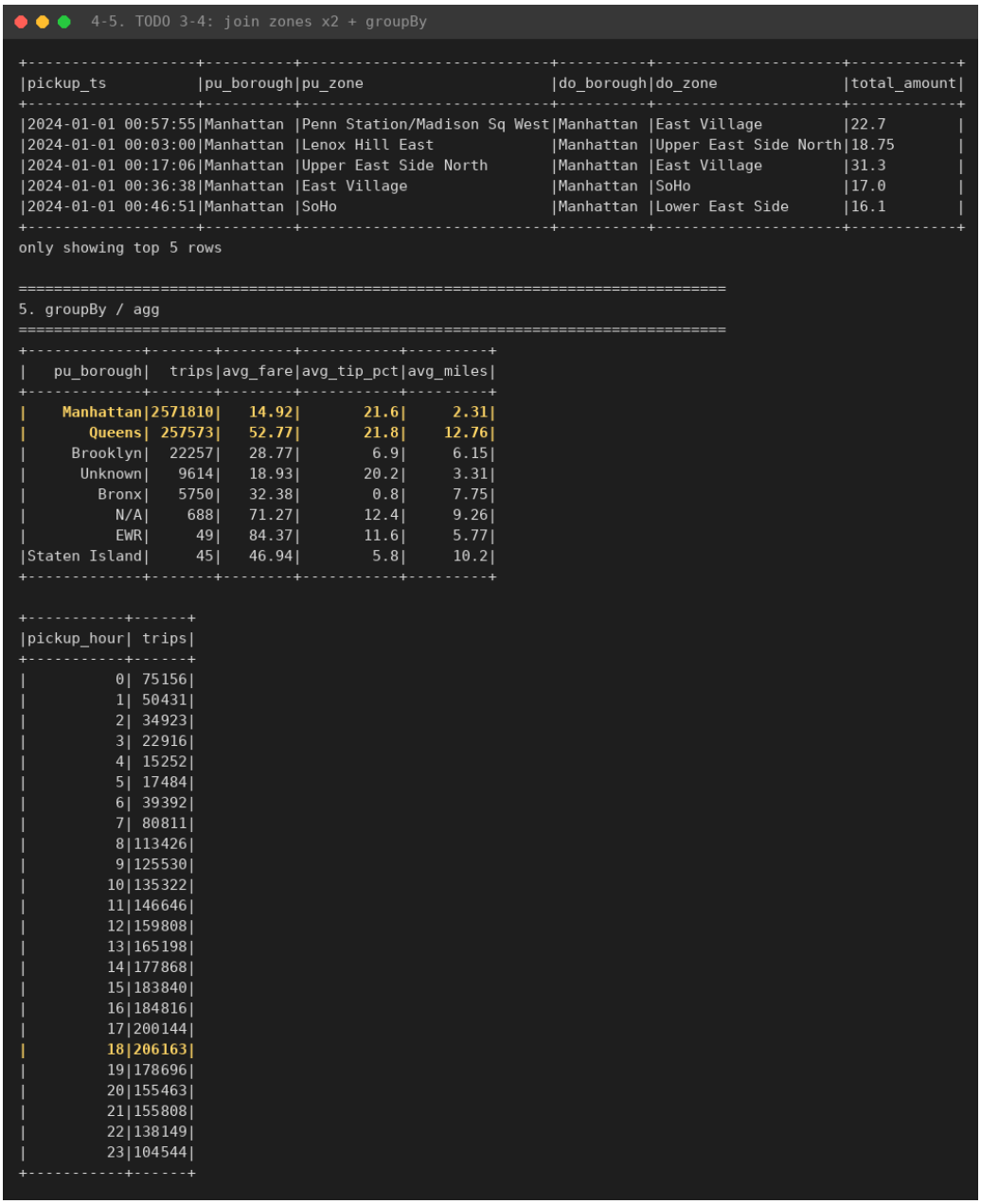
## 2.3 Join với bảng khu vực và tổng hợp

Bảng khu vực được join hai lần, theo điểm đón và điểm trả. Trước mỗi lần join, cột được đổi tên (`PULocationID`/`pu_borough`/`pu_zone` và `DOLocationID`/`do_borough`/`do_zone`); nếu không, hai cột `Borough` trùng tên sẽ gây lỗi `AMBIGUOUS_REFERENCE`. Kết quả ở Hình 5 và Bảng 4.

**Bảng 4.** Tổng hợp theo quận đón sau làm sạch (`by_borough`).

| Quận đón | Số chuyến | Giá cước TB | Tip TB (%) | Quãng đường TB (dặm) |
|---|---|---|---|---|
| Manhattan | 2,571,810 | 14.92 | 21.6 | 2.31 |
| Queens | 257,573 | 52.77 | 21.8 | 12.76 |
| Brooklyn | 22,257 | 28.77 | 6.9 | 6.15 |
| Unknown | 9,614 | 18.93 | 20.2 | 3.31 |
| Bronx | 5,750 | 32.38 | 0.8 | 7.75 |
| N/A | 688 | 71.27 | 12.4 | 9.26 |
| EWR | 49 | 84.37 | 11.6 | 5.77 |
| Staten Island | 45 | 46.94 | 5.8 | 10.2 |

Manhattan chiếm khoảng 89.7 % số chuyến, toàn chuyến ngắn và rẻ. Giá cước trung bình ở Queens cao gấp khoảng 3.5 lần Manhattan, dễ hiểu vì JFK và LaGuardia đều nằm ở Queens. Tip ở Bronx chỉ 0.8 %; chúng tôi đoán phần lớn tiền boa ở đây trả bằng tiền mặt nên không được ghi nhận, nhưng dữ liệu không đủ để khẳng định. Theo giờ đón, khung 18:00 đông nhất với 206,163 chuyến, khung 04:00 vắng nhất với 15,252 chuyến.




**Hình 5.** Kết quả join hai lần với bảng khu vực, tổng hợp theo quận và theo giờ đón.

## 2.4 Hàm cửa sổ

Top-3 khu vực đón mỗi quận dùng `dense_rank()` trên cửa sổ `partitionBy("pu_borough")` (Bảng 5). Riêng Staten Island trả về hơn 3 dòng vì `dense_rank` để nhiều khu vực đồng hạng: ba khu vực cùng 4 chuyến ở hạng 2 và bốn khu vực cùng 2 chuyến ở hạng 3.

**Bảng 5.** Top-3 khu vực đón tại Manhattan và Queens.

| Quận | Hạng 1 | Hạng 2 | Hạng 3 |
|---|---|---|---|
| Manhattan | Midtown Center (140,065) | Upper East Side South (140,059) | Upper East Side North (133,891) |
| Queens | JFK Airport (138,296) | LaGuardia Airport (87,657) | East Elmhurst (11,861) |

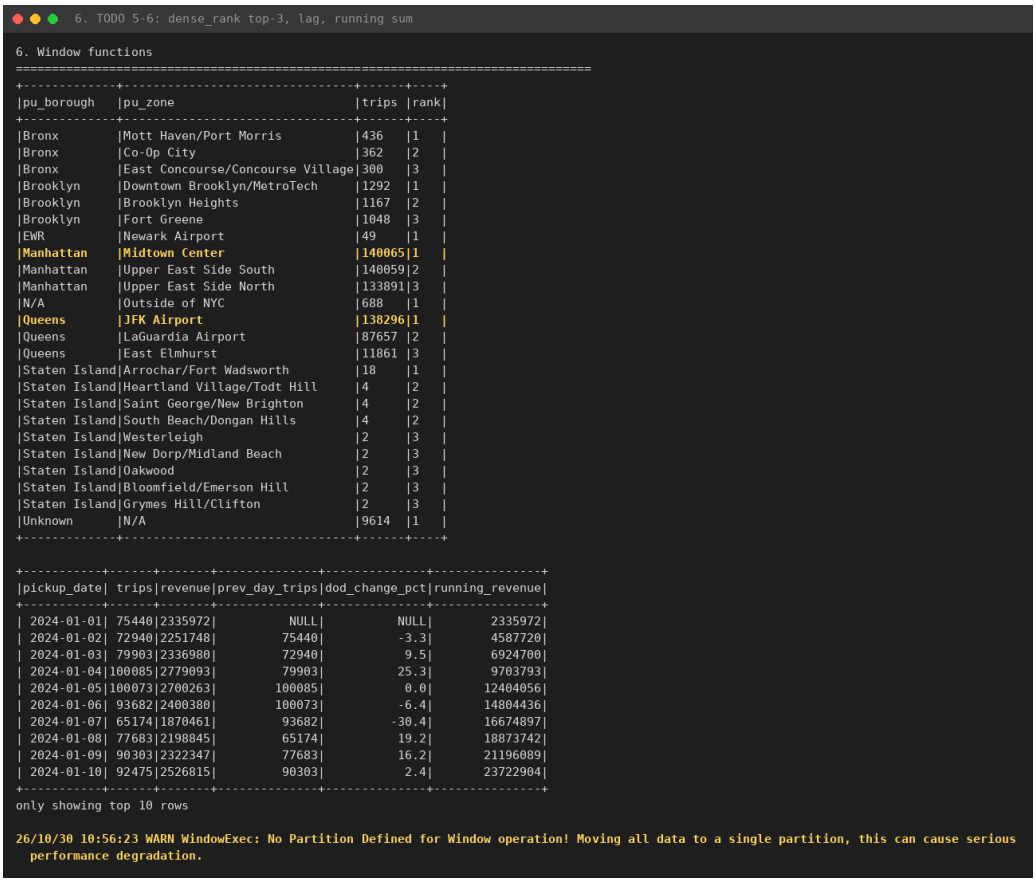
Chuỗi theo ngày dùng `lag` và tổng lũy kế trên `Window.orderBy("pickup_date")` (Bảng 6, Hình 6). Ngày 2024-01-07 (Chủ nhật) giảm 30.4 % so với hôm trước, còn 2024-01-04 tăng 25.3 %.

**Bảng 6.** Số chuyến và doanh thu theo ngày với `lag` và tổng lũy kế (10 ngày đầu).

| Ngày | Số chuyến | Doanh thu | Ngày trước | Thay đổi (%) | Doanh thu lũy kế |
|---|---|---|---|---|---|
| 2024-01-01 | 75,440 | 2,335,972 | NULL | NULL | 2,335,972 |
| 2024-01-02 | 72,940 | 2,251,748 | 75,440 | -3.3 | 4,587,720 |
| 2024-01-03 | 79,903 | 2,336,980 | 72,940 | 9.5 | 6,924,700 |
| 2024-01-04 | 100,085 | 2,779,093 | 79,903 | 25.3 | 9,703,793 |
| 2024-01-05 | 100,073 | 2,700,263 | 100,085 | 0.0 | 12,404,056 |
| 2024-01-06 | 93,682 | 2,400,380 | 100,073 | -6.4 | 14,804,436 |
| 2024-01-07 | 65,174 | 1,870,461 | 93,682 | -30.4 | 16,674,897 |
| 2024-01-08 | 77,683 | 2,198,845 | 65,174 | 19.2 | 18,873,742 |
| 2024-01-09 | 90,303 | 2,322,347 | 77,683 | 16.2 | 21,196,089 |
| 2024-01-10 | 92,475 | 2,526,815 | 90,303 | 2.4 | 23,722,904 |

Log in ra cả chục dòng cảnh báo *“No Partition Defined for Window operation! Moving all data to a single partition”*. Lúc đầu chúng tôi tưởng đó là lỗi, nhưng ở đây cảnh báo vô hại vì cửa sổ chỉ áp lên `daily` đã tổng hợp còn 31 dòng. Áp cùng cửa sổ lên 2.87 triệu chuyến gốc thì toàn

bộ dữ liệu dồn vào một task, mất song song và có thể tràn bộ nhớ.




**Hình 6.** Hàm cửa sổ: top-3 khu vực đón mỗi quận và chuỗi theo ngày với `lag`, tổng lũy kế.

## 2.5 Spark SQL và kế hoạch vật lý

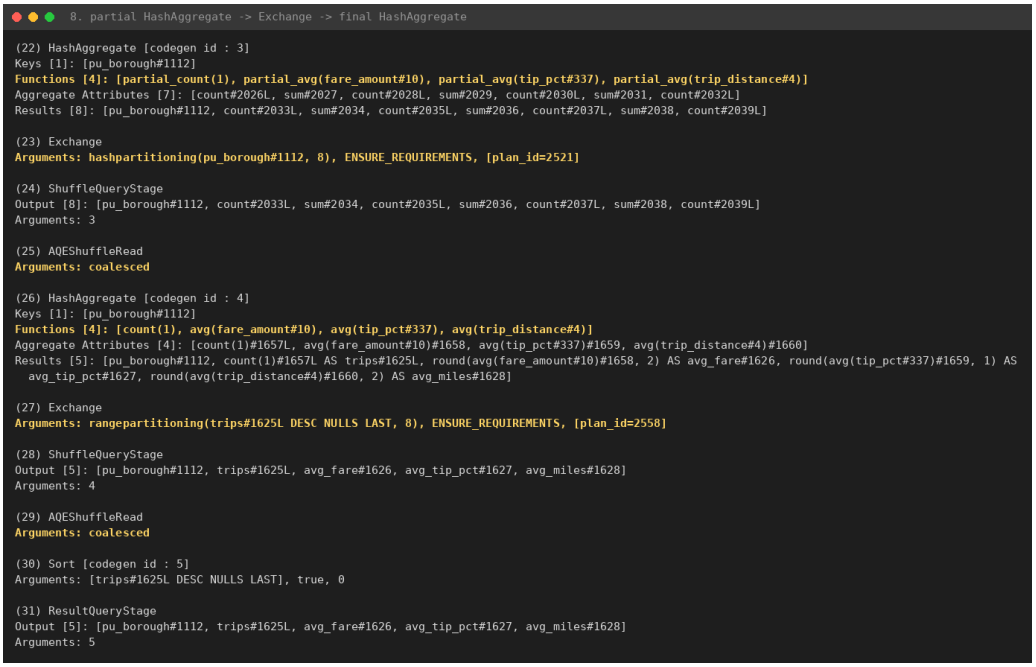
Truy vấn SQL trên temp view `trips` cho đúng bảng ở Bảng 4, script in ra `SQL result == DataFrame result: True` (Hình 7). Hai API đi qua cùng bộ tối ưu Catalyst [1] nên sinh cùng kế hoạch vật lý.




**Hình 7.** Kết quả Spark SQL trùng khớp DataFrame và cây kế hoạch vật lý của `by_borough`.

Kế hoạch `explain("formatted")` của `by_borough` (Hình 7, Hình 8) đọc từ dưới lên như sau. `Scan parquet` (3) mang `PushedFilters: [IsNotNull(tpep_pickup_datetime), IsNotNull(fare_amount), IsNotNull(total_amount), IsNotNull(trip_distance), GreaterThan(fare_amount,0.0), GreaterThanOrEqual(total_amount,0.0), GreaterThan(trip_distance,0.0), LessThanOrEqual(trip_distance,100.0)]`, nghĩa là các luật làm sạch đơn giản đã được đẩy xuống reader Parquet. Luật theo tháng và theo thời lượng cần `date_format`, `unix_timestamp` nên vẫn nằm ở nút `Filter` (5). `InMemoryTableScan` (1) đọc từ cache với `rowCount=2.87E+6`. Hai nhánh `Scan csv` đi qua `BroadcastExchange` (`Statistics(sizeInBytes=1026.1 KiB, rowCount=265)`) vào `BroadcastHashJoin LeftOuter BuildRight` (13) và (20). `HashAggregate` (22) với `partial_count(1), partial_avg(...)` phát ra các cặp sum/count, qua `Exchange hashpartitioning(pu_borough, 8)` (23) có `ShuffleQueryStage` thống kê `sizeInBytes=1968.0 B, rowCount=24`, qua `AQEShuffleRead coalesced` (25), tức Adaptive Query

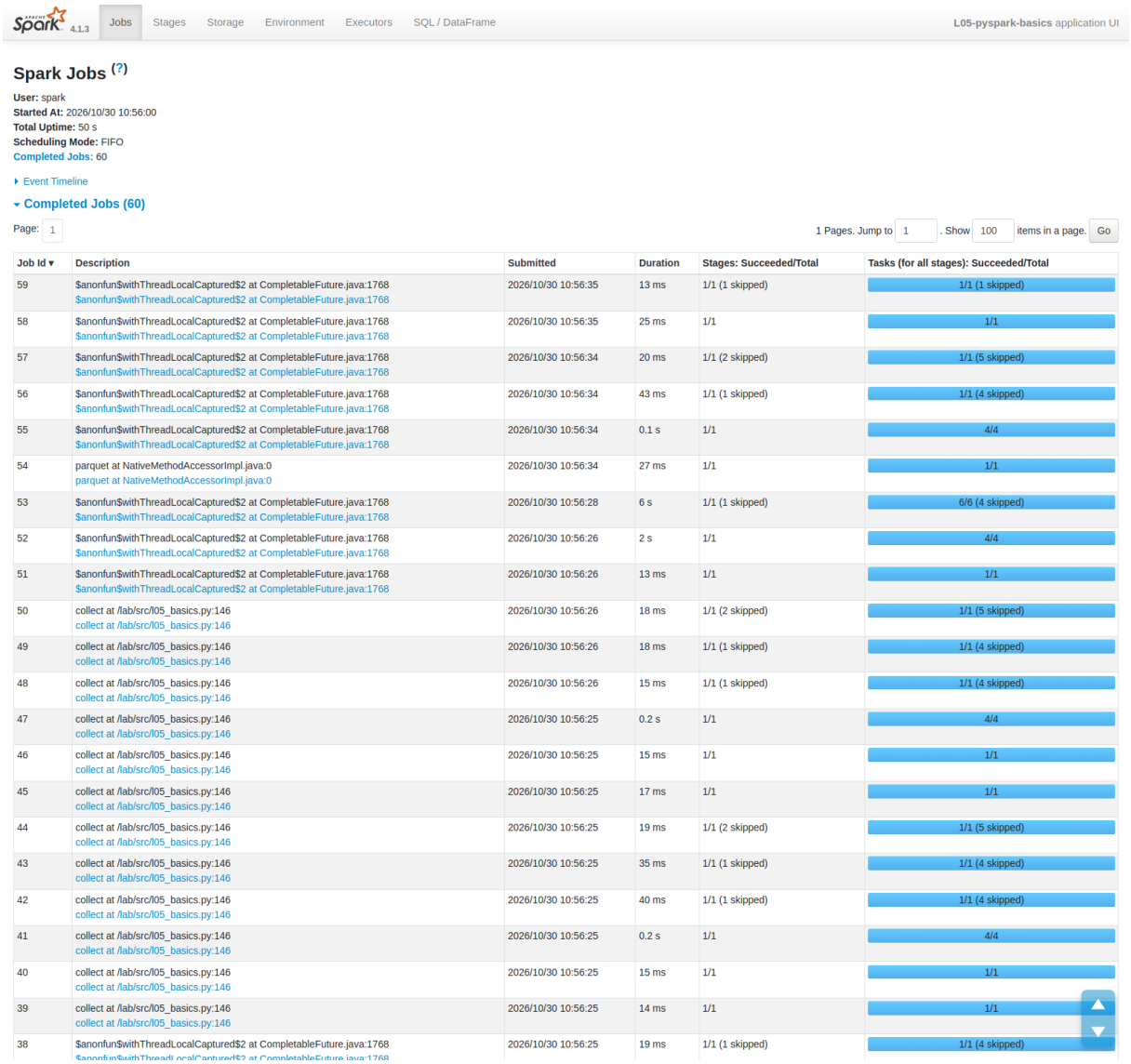
Execution gộp các partition shuffle nhỏ dựa trên thống kê lúc chạy [2], rồi tới `HashAggregate` (26) final. Cuối cùng là `Exchange rangepartitioning(trips DESC NULLS LAST, 8)` (27) với `rowCount=8`, `AQEShuffleRead` (29) và `Sort` (30) cho `orderBy`.



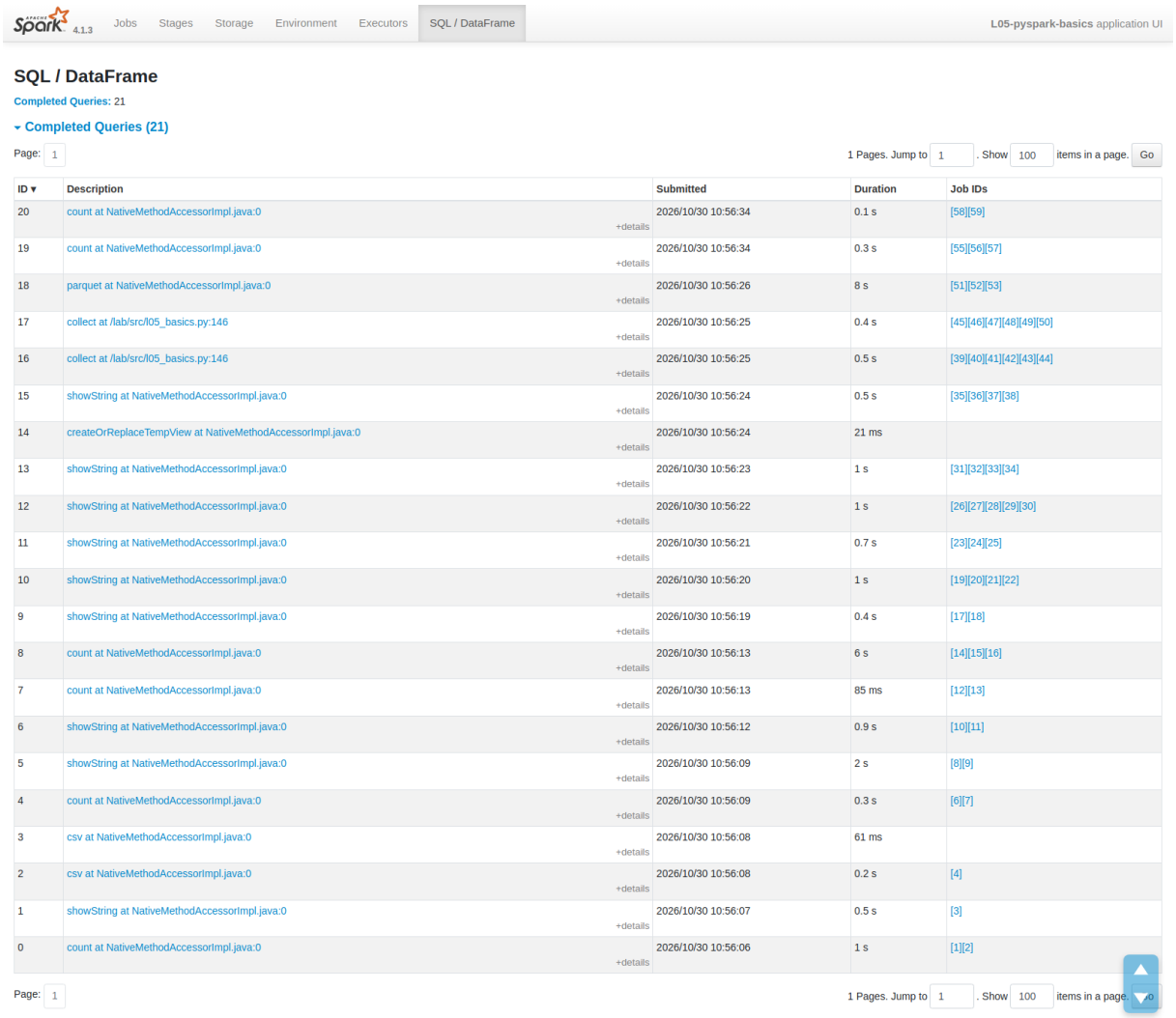

**Hình 8.** Phần kế hoạch thể hiện `HashAggregate` partial, `Exchange hashpartitioning` và `HashAggregate` final.

## 2.6 Spark UI

Trong tab Jobs (Hình 9), mỗi action (`count`, `showString`, `collect`, `parquet`) sinh một hoặc nhiều job. Tab SQL/DataFrame (Hình 10) liệt kê 21 truy vấn; Bảng 7 chọn ra các truy vấn đáng xem.




**Hình 9.** Tab Jobs: mỗi action sinh một hoặc nhiều job.




**Hình 10.** Tab SQL/DataFrame: 21 truy vấn đã hoàn thành cùng thời lượng và danh sách job.

**Bảng 7.** Một số truy vấn trong tab SQL (log `ui_sql_queries.txt`).

| Query | Action | Thời lượng (ms) | Ý nghĩa |
|---|---|---|---|
| 0 | `count` | 1,477 | đếm số dòng đầu vào |
| 4 | `count` | 268 | `count()` ở bước lazy evaluation (228,254 chuyến dài) |
| 8 | `count` | 6,225 | `clean.count()` lần đầu, materialize cache |
| 10 | `showString` | 1,473 | `by_borough.show()` |

| Query | Action | Thời lượng (ms) | Ý nghĩa |
|---|---|---|---|
| 16, 17 | `collect` | 511, 418 | so sánh SQL và DataFrame |
| 18 | `parquet` | 8,127 | ghi Parquet phân vùng có `repartition` |
| 19 | `count` | 336 | đọc lại ngày 2024-01-15 |

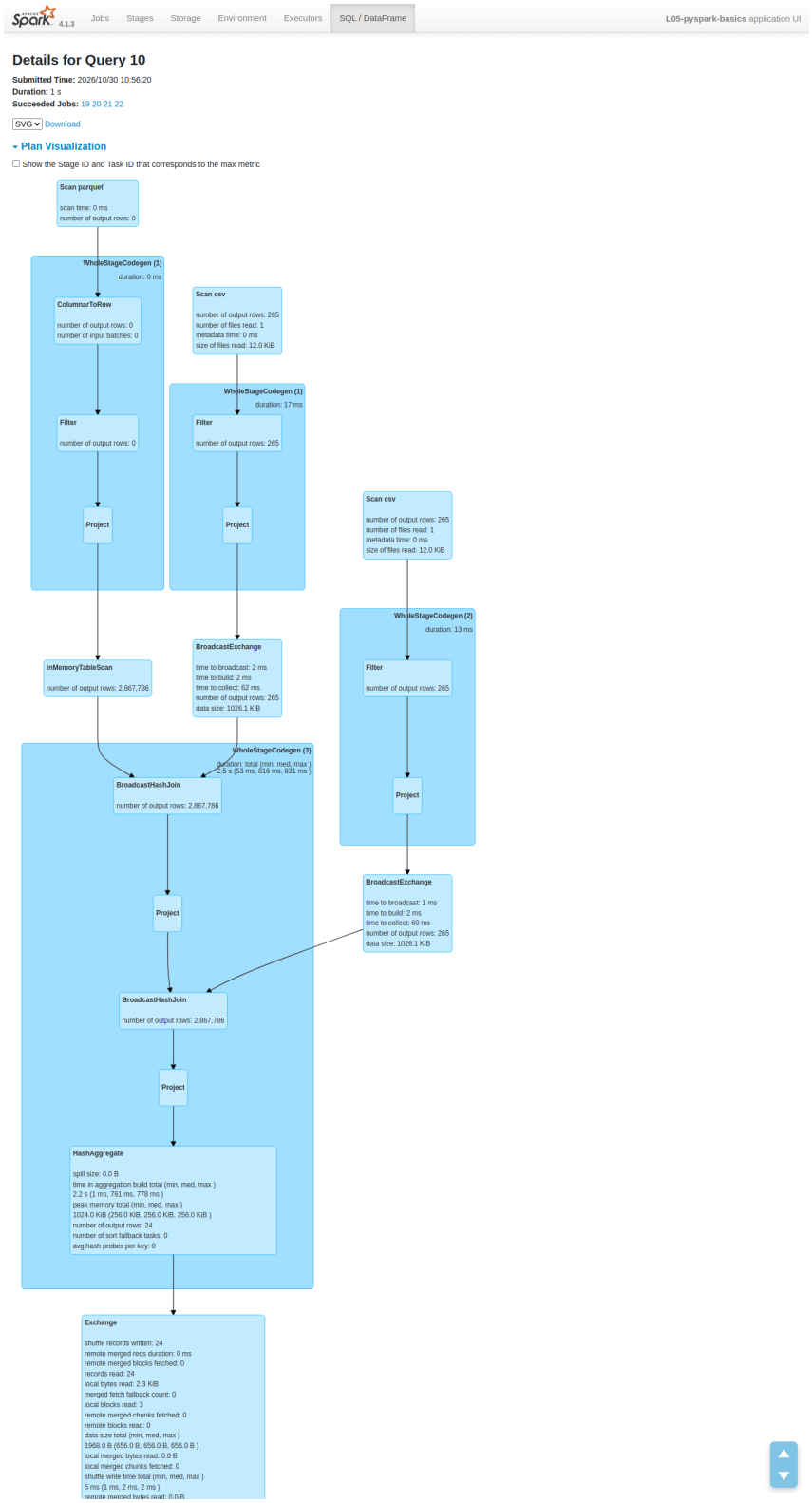
Truy vấn 10 (`by_borough.show()`) sinh bốn job 19–22 (Bảng 8, lấy từ `ui_by_borough_jobs.txt`).

**Bảng 8.** Các job và stage của truy vấn 10 (`by_borough.show()`).

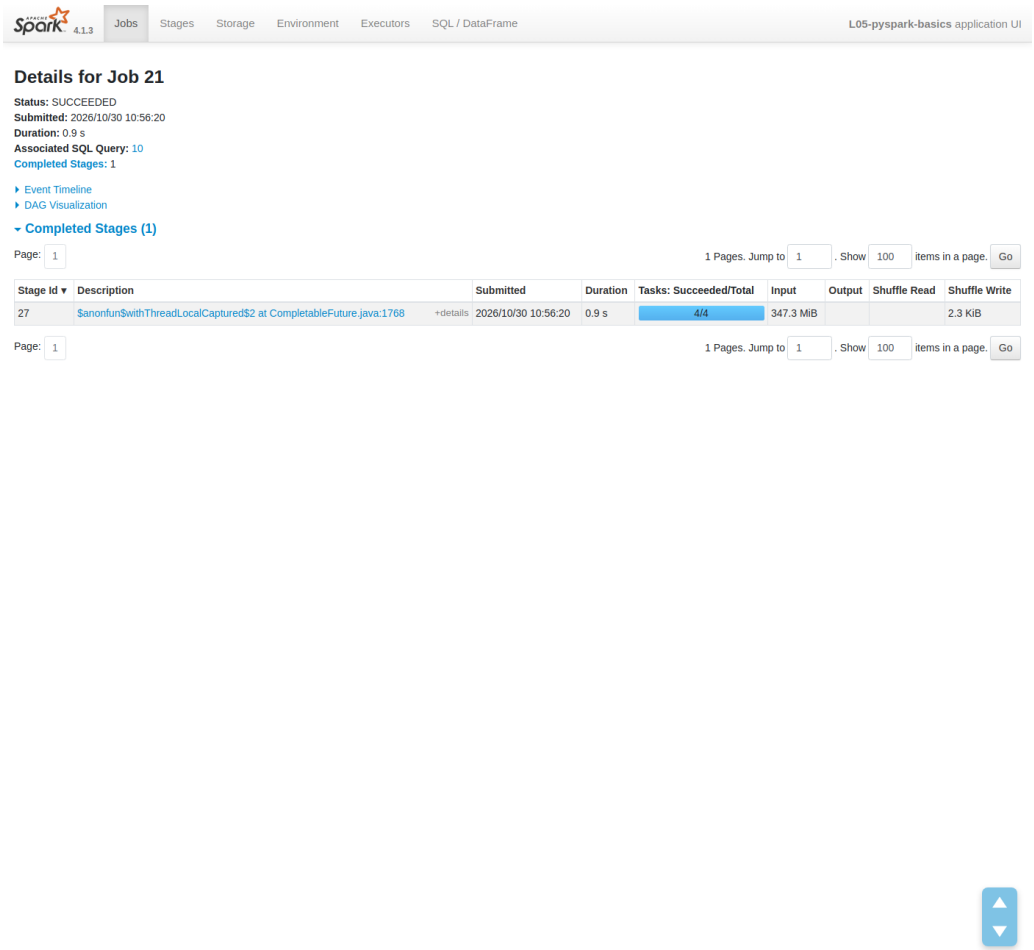
| Job | Stage | Số task | Công việc |
|---|---|---|---|
| 19 | 25 | 1 | dựng broadcast bảng khu vực cho điểm đón (`BroadcastExchange`) |
| 20 | 26 | 1 | dựng broadcast bảng khu vực cho điểm trả |
| 21 | 27 | 4 | `InMemoryTableScan`, 2 × `BroadcastHashJoin`, `HashAggregate` partial, ghi shuffle |
| 22 | 28 (skipped), 29 | 1 | `AQEShuffleRead coalesced`, `HashAggregate` final, `TakeOrderedAndProject` |

Trên đồ thị của truy vấn 10 (Hình 11), `InMemoryTableScan` xuất 2,867,786 dòng, mỗi nhánh `BroadcastExchange` 265 dòng (1026.1 KiB), hai `BroadcastHashJoin` giữ nguyên số dòng, còn `HashAggregate` partial chỉ xuất 24 dòng và `Exchange` ghi 24 bản ghi shuffle (1968.0 B). Job 21 (Hình 12) chạy 4/4 task trong 0.9 s, đọc 347.3 MiB từ cache và chỉ ghi 2.3 KiB shuffle.

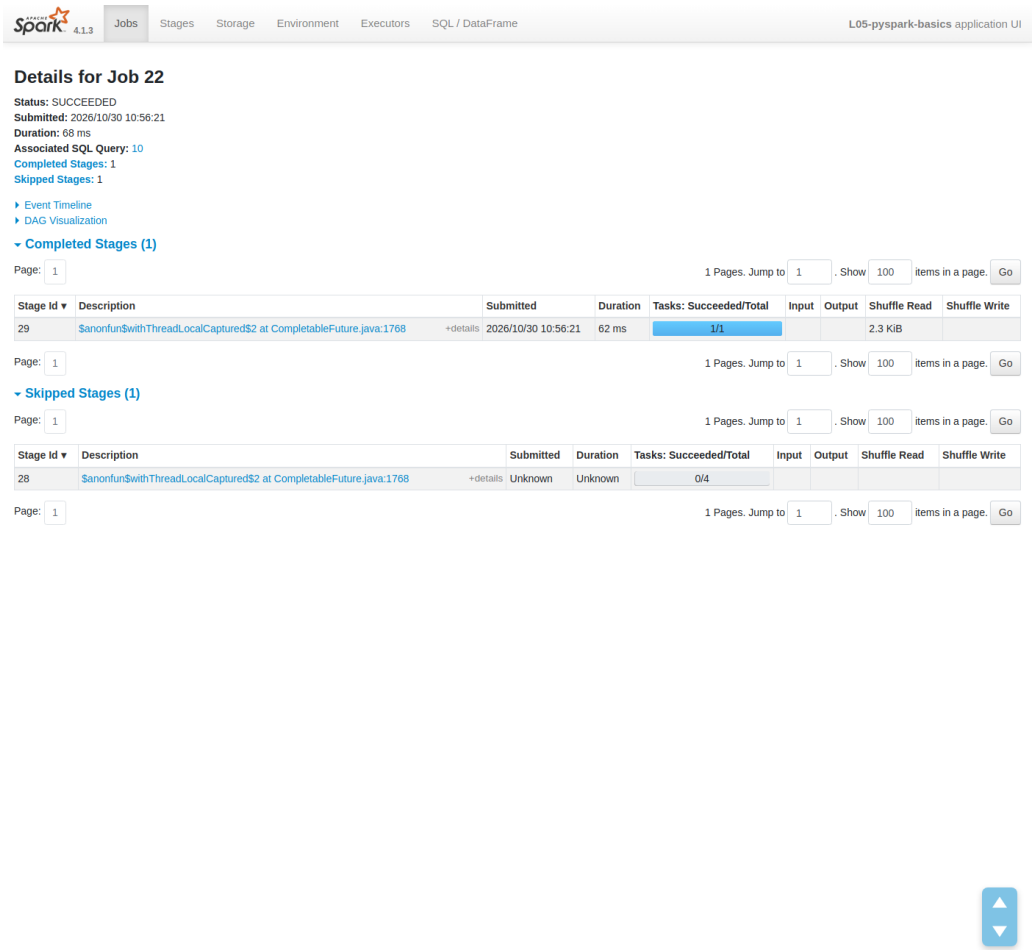
Ở job 22 (Hình 13), stage 28 bị skipped (0/4 task, vì output shuffle của stage 27 đã có sẵn), stage 29 chạy 1 task trong 62 ms với Shuffle Read 2.3 KiB.




**Hình 11.** Đồ thị kế hoạch truy vấn 10 trong tab SQL: 2,867,786 dòng vào, 24 dòng ra khỏi `HashAggregate` partial.

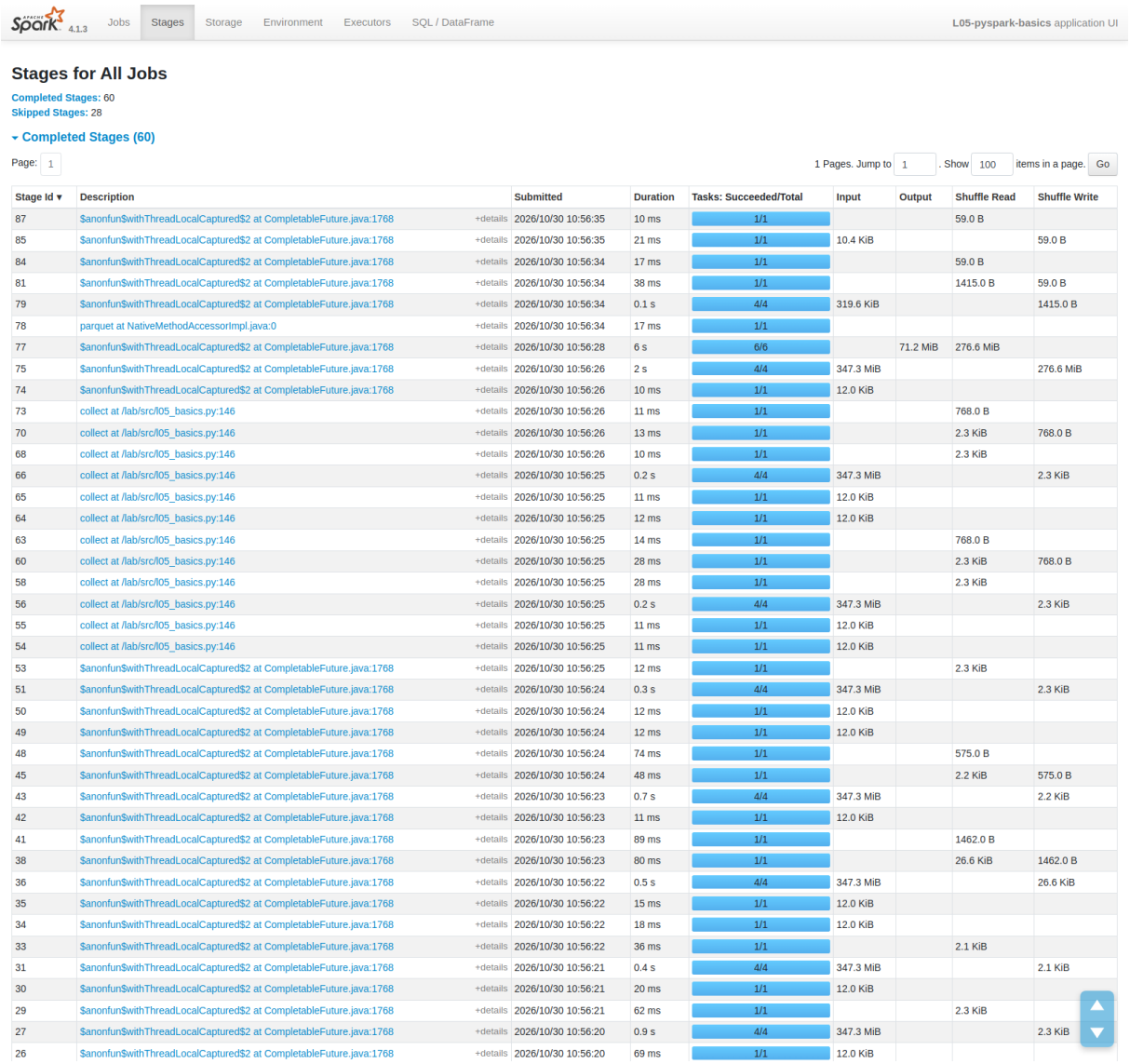



**Hình 12.** Job 21: một stage, 4/4 task, Input 347.3 MiB từ cache, Shuffle Write 2.3 KiB.



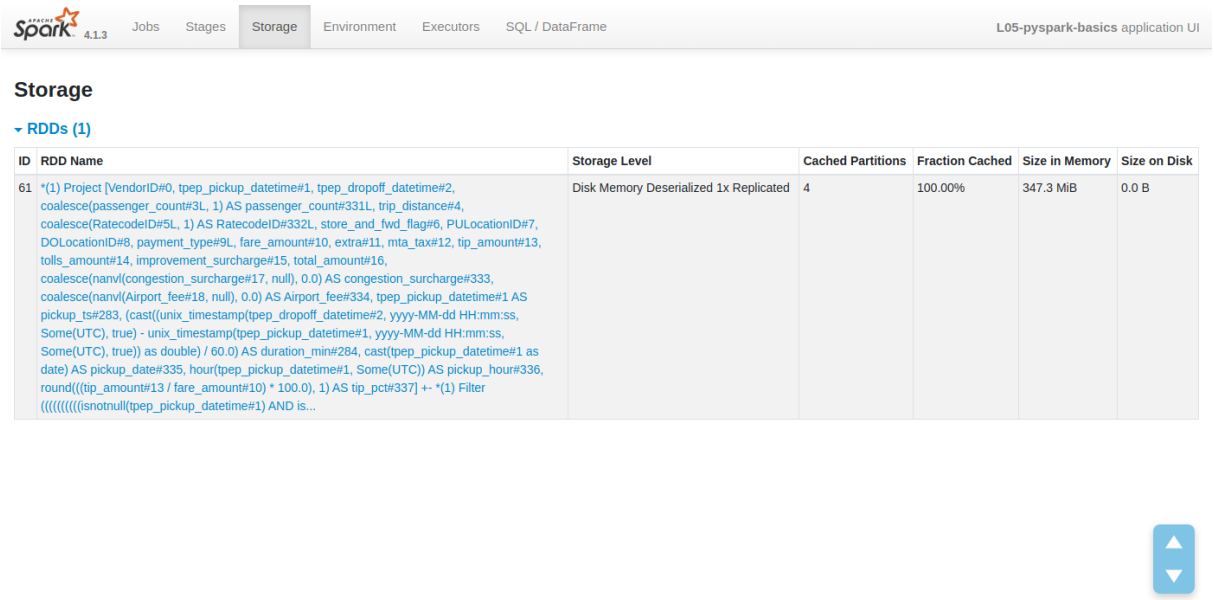

**Hình 13.** Job 22: stage 28 bị bỏ qua, stage 29 chạy 1 task với Shuffle Read 2.3 KiB.

Tab Stages (Hình 14) có 60 stage hoàn thành và 28 stage bị bỏ qua. Các stage 4 task đọc 347.3 MiB từ cache, các stage 1 task đọc 12.0 KiB từ tệp CSV khu vực. Tab Storage (Hình 15) cho thấy `clean` được cache ở mức *Disk Memory Deserialized 1x Replicated*, 4/4 partition (100 %), 347.3 MiB trong bộ nhớ, 0.0 B trên đĩa.




| Stage Id ▾ | Description | Submitted | Duration | Tasks: Succeeded/Total | Input | Output | Shuffle Read | Shuffle Write |
|---|---|---|---|---|---|---|---|---|
| 87 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:35 | 10 ms | 1/1 | | | 59.0 B | |
| 85 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:35 | 21 ms | 1/1 | 10.4 KiB | | | 59.0 B |
| 84 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:34 | 17 ms | 1/1 | | | 59.0 B | |
| 81 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:34 | 38 ms | 1/1 | | | 1415.0 B | 59.0 B |
| 79 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:34 | 0.1 s | 4/4 | 319.6 KiB | | | 1415.0 B |
| 78 | parquet at NativeMethodAccessorImpl.java:0 +details | 2026/10/30 10:56:34 | 17 ms | 1/1 | | | | |
| 77 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:28 | 6 s | 6/6 | | 71.2 MiB | 276.6 MiB | |
| 75 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:26 | 2 s | 4/4 | 347.3 MiB | | | 276.6 MiB |
| 74 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:26 | 10 ms | 1/1 | 12.0 KiB | | | |
| 73 | collect at /lab/src/l05_basics.py:146 +details | 2026/10/30 10:56:26 | 11 ms | 1/1 | | | 768.0 B | |
| 70 | collect at /lab/src/l05_basics.py:146 +details | 2026/10/30 10:56:26 | 13 ms | 1/1 | | | 2.3 KiB | 768.0 B |
| 68 | collect at /lab/src/l05_basics.py:146 +details | 2026/10/30 10:56:26 | 10 ms | 1/1 | | | 2.3 KiB | |
| 66 | collect at /lab/src/l05_basics.py:146 +details | 2026/10/30 10:56:25 | 0.2 s | 4/4 | 347.3 MiB | | | 2.3 KiB |
| 65 | collect at /lab/src/l05_basics.py:146 +details | 2026/10/30 10:56:25 | 11 ms | 1/1 | 12.0 KiB | | | |
| 64 | collect at /lab/src/l05_basics.py:146 +details | 2026/10/30 10:56:25 | 12 ms | 1/1 | 12.0 KiB | | | |
| 63 | collect at /lab/src/l05_basics.py:146 +details | 2026/10/30 10:56:25 | 14 ms | 1/1 | | | 768.0 B | |
| 60 | collect at /lab/src/l05_basics.py:146 +details | 2026/10/30 10:56:25 | 28 ms | 1/1 | | | 2.3 KiB | 768.0 B |
| 58 | collect at /lab/src/l05_basics.py:146 +details | 2026/10/30 10:56:25 | 28 ms | 1/1 | | | 2.3 KiB | |
| 56 | collect at /lab/src/l05_basics.py:146 +details | 2026/10/30 10:56:25 | 0.2 s | 4/4 | 347.3 MiB | | | 2.3 KiB |
| 55 | collect at /lab/src/l05_basics.py:146 +details | 2026/10/30 10:56:25 | 11 ms | 1/1 | 12.0 KiB | | | |
| 54 | collect at /lab/src/l05_basics.py:146 +details | 2026/10/30 10:56:25 | 11 ms | 1/1 | 12.0 KiB | | | |
| 53 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:25 | 12 ms | 1/1 | | | 2.3 KiB | |
| 51 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:24 | 0.3 s | 4/4 | 347.3 MiB | | | 2.3 KiB |
| 50 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:24 | 12 ms | 1/1 | 12.0 KiB | | | |
| 49 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:24 | 12 ms | 1/1 | 12.0 KiB | | | |
| 48 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:24 | 74 ms | 1/1 | | | 575.0 B | |
| 45 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:24 | 48 ms | 1/1 | | | 2.2 KiB | 575.0 B |
| 43 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:23 | 0.7 s | 4/4 | 347.3 MiB | | | 2.2 KiB |
| 42 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:23 | 11 ms | 1/1 | 12.0 KiB | | | |
| 41 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:23 | 89 ms | 1/1 | | | 1462.0 B | |
| 38 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:23 | 80 ms | 1/1 | | | 26.6 KiB | 1462.0 B |
| 36 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:22 | 0.5 s | 4/4 | 347.3 MiB | | | 26.6 KiB |
| 35 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:22 | 15 ms | 1/1 | 12.0 KiB | | | |
| 34 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:22 | 18 ms | 1/1 | 12.0 KiB | | | |
| 33 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:22 | 36 ms | 1/1 | | | 2.1 KiB | |
| 31 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:21 | 0.4 s | 4/4 | 347.3 MiB | | | 2.1 KiB |
| 30 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:21 | 20 ms | 1/1 | 12.0 KiB | | | |
| 29 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:21 | 62 ms | 1/1 | | | 2.3 KiB | |
| 27 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:20 | 0.9 s | 4/4 | 347.3 MiB | | | 2.3 KiB |
| 26 | $anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details | 2026/10/30 10:56:20 | 69 ms | 1/1 | 12.0 KiB | | | |

**Hình 14.** Tab Stages: 60 stage hoàn thành, 28 stage bị bỏ qua, kèm Input và Shuffle Read/Write.




**Hình 15.** Tab Storage: `clean` được cache 4/4 partition, 100 %, 347.3 MiB trong bộ nhớ.

## 2.7 Ghi Parquet phân vùng và partition pruning

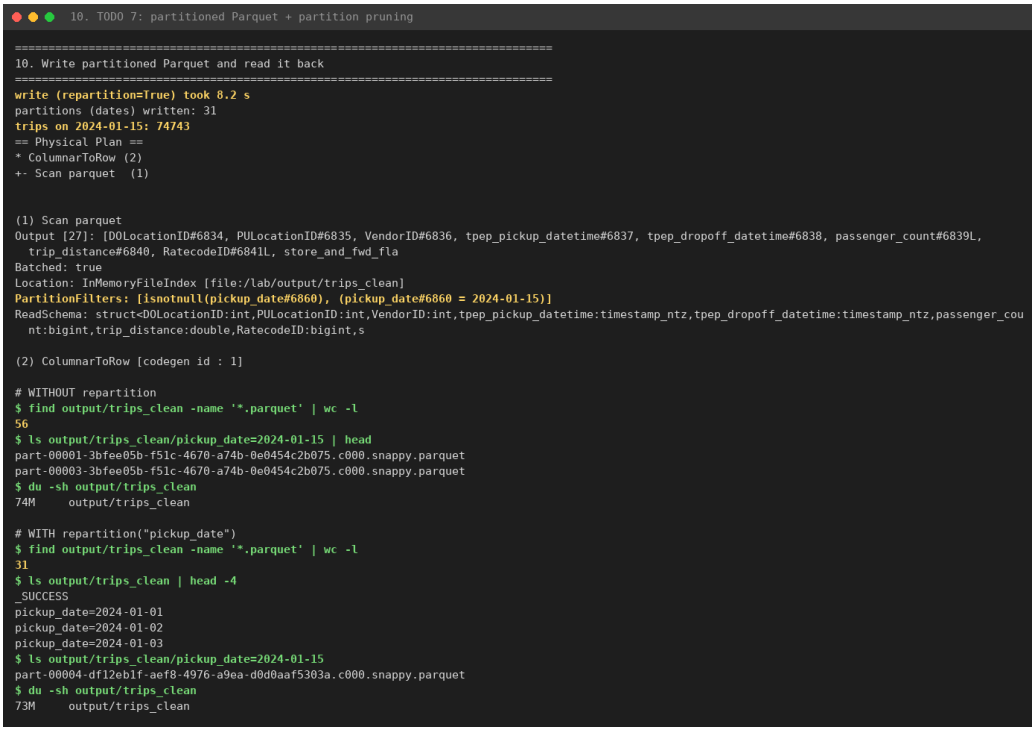
`enriched` được ghi bằng `partitionBy("pickup_date")`, một lần không và một lần có `repartition("pickup_date")` (Bảng 9, theo `02_files_no_repartition.txt`, `04_files_with_repartition.txt` và hai log chạy).

**Bảng 9.** So sánh ghi Parquet phân vùng theo ngày có và không có `repartition`.

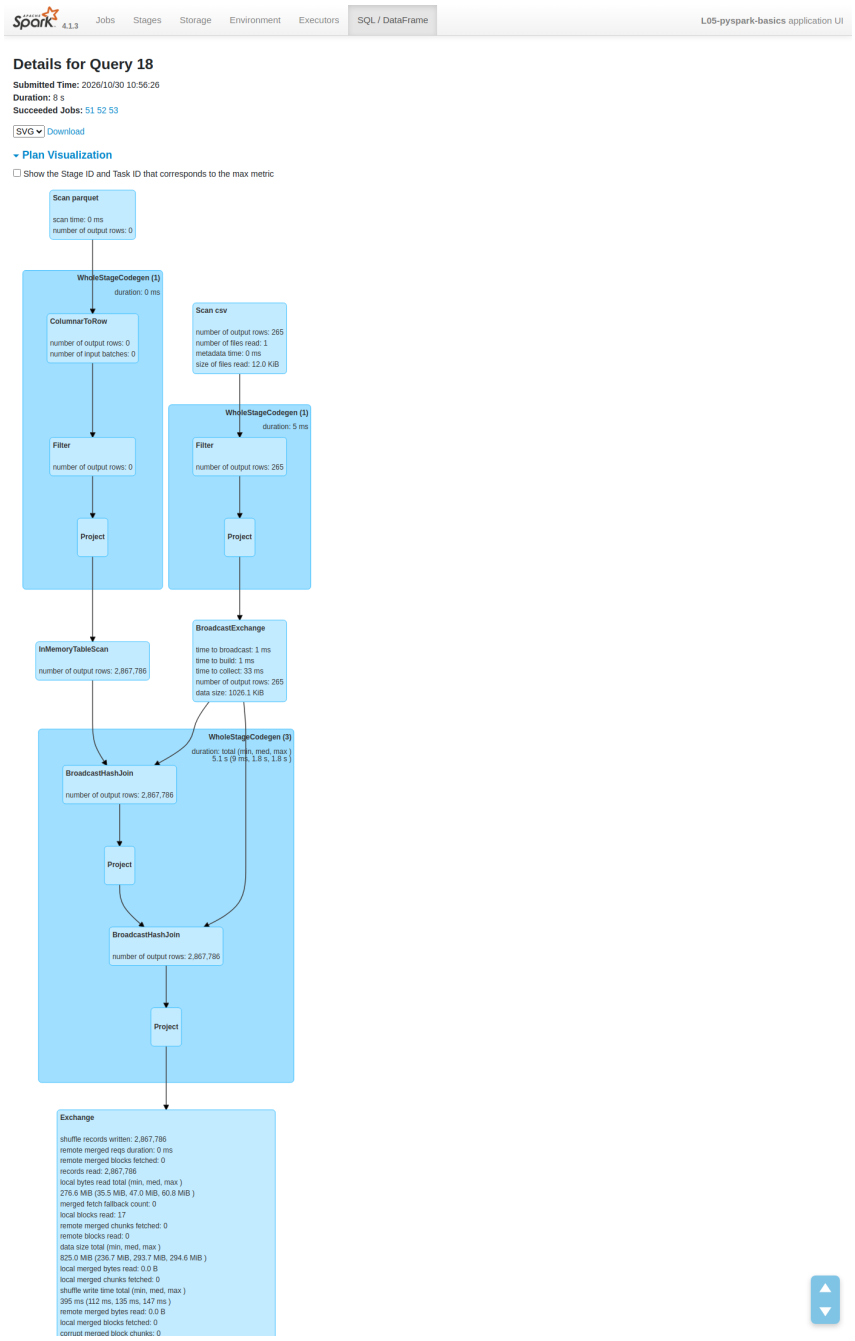
| Chỉ số | Không `repartition` | `repartition("pickup_date")` |
|---|---|---|
| Số tệp `.parquet` | 56 | 31 (1 tệp/ngày) |
| Số tệp trong `pickup_date=2024-01-15` | 2 (`part-00001`, `part-00003`) | 1 (`part-00004`) |
| Dung lượng thư mục output | 74 MB | 73 MB |
| Thời gian ghi | 6.8 s | 8.2 s |
| Tổng thời gian chạy script | 35.70 s | không so sánh được (có `--hold 300`) |

Cả hai lần đều ra 31 thư mục ngày. Đọc lại với `filter(pickup_date == '2024-01-15')` trả về 74,743 chuyến, và nút `Scan parquet` có `PartitionFilters: [isnotnull(pickup_date), (pickup_date = 2024-01-15)]` (Hình 16), tức chỉ một thư mục được mở. Ở đồ thị truy vấn 18 (Hình 17), `repartition` thêm một `Exchange` mà cả 2,867,786 dòng phải đi qua; trong tab Stages, stage 75 ghi 276.6 MiB shuffle, stage 77 (6 task sau khi AQE gộp partition)

đọc lại lượng đó và xuất 71.2 MiB Parquet (Hình 14). Mỗi cấu hình chỉ chạy một lần, nên chênh lệch thời gian ghi mới là một mẫu, chưa đủ để kết luận chắc về chi phí của `repartition`.




**Hình 16.** Ghi Parquet phân vùng, đọc lại ngày 2024-01-15 (74,743 chuyến) và `PartitionFilters` trong kế hoạch.




**Hình 17.** Đồ thị truy vấn 18 (ghi Parquet): `Exchange` do `repartition` shuffle toàn bộ 2,867,786 dòng.

# 3 Trả lời câu hỏi checkpoint

## 3.1 Vì sao tệp 48 MB lại có 4 partition đầu vào?

Spark cắt tệp thành các split có kích thước tối đa maxSplitBytes $=$ min(maxPartitionBytes, max(openCostIn với `spark.sql.files.maxPartitionBytes` mặc định 128 MB, `openCostInBytes` mặc định 4 MB, còn `bytesPerCore` là tổng dung lượng (cộng chi phí mở tệp) chia cho `defaultParallelism`. Ở `local[4]`, `defaultParallelism` bằng 4 nên split bị hạ xuống khoảng 48 MB / 4 thay vì 128 MB, ra 4 split và 4 partition (Hình 2). Có điều Parquet chỉ cắt

được ở ranh giới row group: task nào nhận điểm bắt đầu của row group thì đọc cả row group đó. Chúng tôi không tự kiểm tra số row group, nhưng theo đáp án tham khảo của bài lab tệp này có 3 row group, nên một partition có thể rỗng. Quan sát ở Bảng 9 khớp với điều này: các tệp của ngày 2024-01-15 mang số `part-00001` và `part-00003`, không có task 2.

## 3.2 Dòng nào là transformation, dòng nào là action, và sao dựng plan nhanh vậy?

`filter`, `withColumn`, `select` là transformation: chúng chỉ thêm nút vào kế hoạch logic, không chạm dữ liệu, nên tab Jobs không có job mới (Mục 2.1). `count()` là action: Catalyst phân tích và tối ưu kế hoạch, DAG scheduler tạo stage và task, rồi mới đọc tệp; thời gian đo được khớp với truy vấn 4 trong Bảng 7. `show`, `collect`, `take`, `write` cũng là action. Dựng plan nhanh vì đó chỉ là thao tác trên cây biểu thức trong bộ nhớ driver, không có I/O và không cần liên lạc với executor.

## 3.3 `by_borough.show()` sinh bao nhiêu job, stage, và stage bị cắt ở đâu?

Truy vấn 10 sinh **4 job (19–22) với 5 stage (25–29)**, trong đó stage 28 bị bỏ qua nên chỉ 4 stage thực sự chạy (Bảng 8). Job 22 đọc shuffle đã được AQE gộp từ 8 xuống 1 partition, tổng hợp final và lấy kết quả có thứ tự cho `show()`.

Ranh giới stage nằm ở các `Exchange`: giữa stage 27 và stage 29 là `Exchange hashpartitioning(pu_borough, 8)`. Broadcast phải chạy thành job riêng vì `BroadcastExchange` cần thu dữ liệu về driver rồi phát lại trước khi phía lớn join được. Với AQE, mỗi query stage được nộp như một job riêng để Spark tối ưu lại khi đã có thống kê thật, vì thế stage 28 (bản sao của stage 27 trong DAG của job 22) hiện là skipped (Hình 13). Với `show()`, Spark dùng `TakeOrderedAndProject` thay cho sort toàn cục. Nếu tính trọn `by_borough` (ví dụ `collect`), kế hoạch có thêm `Exchange rangepartitioning(trips DESC, 8)` và `Sort`, tức thêm một stage, như trong `explain` (Hình 7). Trước khi gọi `show()`, các phép join và `groupBy` chưa chạy gì, chúng chỉ dựng DAG.

## 3.4 Vì sao năm cột có cùng 140,162 NULL, và nên xử lý thế nào?

Năm cột trùng số NULL (Hình 3) vì đó là cùng các chuyến: các bản ghi này thiếu cả nhóm trường metadata của chuyến (thường từ một nhà cung cấp, hoặc chuyến đón vẫy tay ngoài đường), chứ không phải năm lỗi độc lập. Nếu độc lập thì gần như không thể trùng số tuyệt đối ở cả năm cột. Có ba cách xử lý: (i) giữ dòng và điền mặc định như script đã làm (Mục 2.2); (ii) giữ NULL và chỉ bỏ các dòng này khỏi những chỉ số cần đến các cột đó; (iii) tách sang vùng quarantine để kiểm tra sau. Chúng tôi chọn (i) cho đơn giản, nhưng điền `passenger_count=1` có thể làm lệch chỉ số số khách trung bình nếu sau này cần tới.

## 3.5 `HashAggregate` partial trước `Exchange` làm gì, ứng với khái niệm nào ở L02?

`HashAggregate` (22) trước `Exchange` là pha partial: mỗi task gộp trước các dòng của mình theo `pu_borough`. Hàm `avg` bị tách thành cặp `(sum, count)` (thấy trong `Aggregate Attributes` gồm `count#...`, `sum#...`), vì trung bình của các trung bình không bằng trung bình chung, còn tổng và số đếm thì cộng dồn được. Mỗi task chỉ phát một dòng cho mỗi quận nó gặp, nên 4 task cộng lại chỉ có **24 dòng** qua shuffle thay vì 2,867,786 dòng (Hình 11). Sau `Exchange` (ranh giới stage), `HashAggregate` (26) final hợp nhất các kết quả partial (Hình 8).

Đây là combiner của MapReduce ở L02 [4]: gộp phía map để giảm dữ liệu trung gian trước khi gửi sang reducer. Việc tách `avg` thành `(sum, count)` giống bài mở rộng L02 (combiner tính giá cước trung bình); lúc đó chúng tôi phải tự nghĩ ra cặp này, còn ở Spark Catalyst tự làm.

## 3.6 Vì sao bảng khu vực được broadcast, và khi nào Spark chuyển sang sort-merge join?

Bảng khu vực chỉ có 265 dòng (tệp 12 KB, khoảng 1 MB trong bộ nhớ theo `Statistics(sizeInBytes= 1026.1 KiB)`), nhỏ hơn nhiều so với ngưỡng `spark.sql.autoBroadcastJoinThreshold` mặc định 10 MB [2]. Spark gửi một bản sao tới mọi task nên 2,867,786 chuyến không phải shuffle; trên Hình 11, `BroadcastHashJoin` nằm chung một khối WholeStageCodegen với phép quét cache. Khi cả hai phía đều vượt ngưỡng, khi ngưỡng đặt bằng -1, hoặc khi thống kê ước lượng vượt ngưỡng, Spark shuffle cả hai phía theo khóa join và dùng SortMergeJoin.

## 3.7 `cache()` thay đổi gì trong plan và UI, và khi nào không nên cache?

Sau `cache()`, kế hoạch đọc từ `InMemoryRelation`/`InMemoryTableScan` thay vì quét và lọc lại Parquet. Nhánh `Scan parquet` vẫn còn trên đồ thị nhưng ra 0 dòng (Hình 11). Tab Storage hiện RDD được cache (Hình 15). Theo Bảng 7, `clean.count()` lần đầu (truy vấn 8) chậm hẳn vì phải materialize cache, còn các truy vấn dùng cache sau đó (9–17) chỉ mất 396 ms đến 1,473 ms.

Không nên cache khi dữ liệu chỉ dùng một lần, hoặc khi nó không vừa bộ nhớ và gây spill, eviction, áp lực GC. Ở đây cache chiếm 347 MiB so với khoảng 48 MB Parquet nén, gấp khoảng 7 lần, nên với dữ liệu cả năm thì driver 2 GB khó mà chứa nổi. Cache cũng có thể thừa khi nguồn đọc vốn đã rẻ: Parquet có column pruning đôi khi nhanh hơn một cache dạng hàng rộng. Cuối cùng, quên `unpersist()` thì bộ nhớ bị giữ vô ích.

### 3.8 Vì sao cần `repartition("pickup_date")` trước `partitionBy("pickup_date")`?

`partitionBy` ghi một tệp cho mỗi cặp (task, giá trị phân vùng) mà task đó chứa. Không `repartition` thì mỗi task có thể chứa dòng của nhiều ngày, số tệp tối đa là $N \times 31$ với $N$ task, ở đây là $4 \times 31 = 124$. Số tệp đo được ở Bảng 9 ít hơn giới hạn khá nhiều, vì tệp nguồn gần như đã sắp theo thời gian: mỗi task chỉ chứa một dải ngày liền nhau và chỉ vài ngày bị tách giữa hai task (như 2024-01-15). Với dữ liệu không có thứ tự và hàng trăm task trên cluster thật, con số sẽ lên tới hàng nghìn tệp nhỏ, gây áp lực cho NameNode và làm chậm việc đọc.

`repartition("pickup_date")` shuffle trước để mọi dòng của một ngày về cùng một task, nên mỗi ngày đúng một tệp. Cái giá là một lần shuffle toàn bộ dữ liệu (Mục 2.7, Hình 17) và thời gian ghi tăng (Bảng 9). Với dữ liệu lệch (skew), một ngày quá lớn sẽ dồn vào một task và sinh một tệp khổng lồ; khi đó có thể dùng `repartition(n, "pickup_date", salt)` hoặc `maxRecordsPerFile`.

## 4 Kết luận

Script chạy đủ các bước của đề và cho đúng các số liệu tham khảo: 2,964,624 dòng trong 4 partition, 228,254 chuyến dài, 2,867,786 dòng giữ lại sau làm sạch, Manhattan 2,571,810 chuyến, giờ cao điểm 18:00, 31 phân vùng ngày và 74,743 chuyến ngày 2024-01-15. Trên log và UI, chúng tôi quan sát được transformation chỉ dựng kế hoạch còn action mới chạy job, stage bị cắt ở `Exchange`, tổng hợp hai pha chỉ đưa 24 dòng qua shuffle, bảng nhỏ được broadcast tự động, và DataFrame với SQL cho cùng kết quả. Ghi có `repartition` cho đúng một tệp mỗi ngày, đổi lại một lần shuffle toàn bộ dữ liệu.

## Tài liệu tham khảo

[1] M. Armbrust *et al.*, “Spark SQL: Relational Data Processing in Spark,” in *Proc. ACM SIGMOD*, 2015.

[2] Apache Software Foundation, “Performance Tuning,” Apache Spark Documentation. https://spark.apache.org/docs/latest/sql-performance-tuning.html

[3] Apache Software Foundation, “ANSI Compliance,” Apache Spark Documentation. https://spark.apache.org/docs/latest/sql-ref-ansi-compliance.html

[4] J. Dean and S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” in *Proc. 6th USENIX OSDI*, 2004.

[5] NYC Taxi and Limousine Commission, “TLC Trip Record Data.” https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

# Phụ lục: Trích đoạn mã nguồn

Đếm NULL của mọi cột trong một lần quét và tính tỷ lệ tip an toàn với ANSI mode (phần làm sạch trong `l05_basics.py`):

```
null_counts = trips.select([F.sum(F.col(c).isNull().cast("int")).alias(c)
                            for c in trips.columns])

# ANSI mode: try_divide trả NULL khi fare_amount = 0 thay vì làm hỏng job
.withColumn("tip_pct",
            F.round(F.try_divide(F.col("tip_amount"), F.col("fare_amount"))
 ↪  * 100, 1))
```

Join hai lần với bảng khu vực, đổi tên cột trước mỗi lần để tránh `AMBIGUOUS_REFERENCE`:

```
pu = zones.select(F.col("LocationID").alias("PULocationID"),
                  F.col("Borough").alias("pu_borough"),
 ↪  F.col("Zone").alias("pu_zone"))
do = zones.select(F.col("LocationID").alias("DOLocationID"),
                  F.col("Borough").alias("do_borough"),
 ↪  F.col("Zone").alias("do_zone"))
enriched = clean.join(pu, "PULocationID", "left").join(do, "DOLocationID",
 ↪  "left")
```

Ghi Parquet phân vùng; cờ `--no-repartition` cho phép chạy cả hai cấu hình mà không sửa mã:

```
out = enriched.drop("pickup_ts")
if not args.no_repartition:
    out = out.repartition("pickup_date")
out.write.mode("overwrite").partitionBy("pickup_date").parquet(OUTPUT)
```